
aliases: - 01-fstring

theory_01: f-string — 把变量塞进字符串

读完约 3 分钟。

相关笔记: [类型转换](#) · [字符串方法](#)

你已经会的两种方式

```
name = "张三"
age = 20

# 方式1: 逗号 (有空格的毛病)
print("我叫", name, ", 今年", age, "岁")

# 方式2: 加号 (数字要手动转str的毛病)
print("我叫" + name + ", 今年" + str(age) + "岁")
```

f-string: 第三种方式

```
print(f"我叫 {name}, 今年 {age} 岁")
```

规则就两条: 1. 引号前写 `f` 2. 变量名放 `{}` 里

Python 自动去 `{}` 里取值, 自动把数字转成字符串。

`{}` 里可以是任意计算

```
a = 10
b = 3

print(f"{a} + {b} = {a + b}")      # 10 + 3 = 13
print(f"{a} 的平方是 {a ** 2}")    # 10 的平方是 100
```

浮点数精度问题

```
price = 29.9
count = 3
print(price * count)                # 89.69999999999999 ← 不是 89.7!

# 修法: {:.1f} 保留1位小数, 自动四舍五入
print(f"总价{price * count:.1f}元") # 总价89.7元
print(f"总价{price * count:.2f}元") # 总价89.70元 (保留2位)
```

金融/价格相关永远加 `{:.2f}`。

为什么会有精度问题 (IEEE 754)

计算机用二进制存小数。十进制里能精确表示的 `0.1`，在二进制里是 `0.0001100110011...` 无限循环，必然被截断。

```
0.1 + 0.2    # 0.30000000000000004 ← 不是 0.3
```

所以 `29.9 * 3 = 89.699...` 不是 Python 的 bug，所有语言都这样。

需要精确计算时用 Decimal

```
from decimal import Decimal

result = Decimal("29.9") * 3    # 89.7, 精确
```

注意：必须用字符串 `"29.9"` 传入 `Decimal`。用 `Decimal(29.9)` 还是会有精度问题——因为 `29.9` 传进去之前就已经被 Python 截断了。

进阶规则：f-string 两条核心规则

规则一： {} 里放的是表达式，不是语句

```
name = "小明"
print(f"{name.upper()}")      # ✅ 表达式，可以
# print(f"{if name: ...}")    # ❌ 语句，不行
```

规则二： ! 控制转换， : 控制格式

```
x = 3.14159
print(f"{x!r}")               # !r → repr(), 输出 '3.14159' (给开发者看的)
print(f"{x:.2f}")             # :.2f → 保留两位小数, 输出 3.14 (给人看的)
```

符号	作用	等价写法
!r	repr() 表示 (调试用)	f"{repr(x)}"
!s	str() 表示 (默认, 可省略)	f"{str(x)}"
!a	ascii() 表示 (转义非ASCII字符)	f"{ascii(x)}"
:.2f	格式说明 (: 开头, 控制精度/对齐/进制)	—

! 和 : 可以同时用: f"{x!r:.10}" — 先 repr 再截断到10字符。

常见坑

```
# 坑1: 忘写 f
print("Hello, {name}!")      # 输出 Hello, {name}! —— 没替换!

# 坑2: f-string 的 {} 和切片的 [] 不是一回事
names = ["张三", "李四"]
print(f"{names[0]}")         # OK: f-string {} 里用索引
# print(f"{names[:]}")       # 不报错但输出 ['张三','李四'], 不是你要的格式
```

引号搭配

```
print(f"他说: {'你好'}")    # OK: 外面双引号, {}里面单引号
# print(f"他说: {"你好"}")  # 报错! 内外同种引号冲突
```

enumerate: 循环带编号

```
names = ["张三", "李四", "王五", "赵六"]

for i, name in enumerate(names, 1):    # 第二个参数1 = 从1开始编号
    print(f"第{i}位: {name}")

# 第1位: 张三 第2位: 李四 第3位: 王五 第4位: 赵六
```

实战: 三种格式化方式 (你微信收到的学习文件)

1. % 格式化 → 格式化字符串%.py

```
name = '张三'
age = 27
ave = 88.856
address = '北京昌平区'
print("姓名: %s" % name)
print("年龄: %6d岁\n家庭住址: %s\n平均成绩%.2f" % (age, address, ave))
```

要点: %s = 字符串, %d = 整数, %f = 浮点数, %.2f = 保留2位小数, %6d = 宽度6右对齐。

2. format() 方法 → 格式化字符串format方法.py

```
name = '张三'
age = 27
address = '北京昌平区'
```

```
print("姓名: {}".format(name))          # 默认
print("年龄: {}岁\n家庭住址: {}".format(age, address)) # 顺序
print("姓名: {1}\n年龄: {0}".format(age, name))      # 编号
print("姓名: {name1}\n年龄: {age1}".format(name1=name, age1=age)) # 名称

point, total = 19, 22
print('所占百分比: {:.2%}'.format(point/total))    # 百分比: 86.36%
```

要点： `{}` 是占位符，支持顺序/编号/名称三种定位方式， `:.2%` 自动转百分比。

3. f-string → 格式化字符串f-string.py

```
age = 20
ave_score = 88.8534
gender = '男'
print(f"年龄: {age}, 性别: {gender}, 平均成绩: {ave_score:.2f}")
```

要点： 最简洁的写法， `f"..."` 里用 `{变量名}` 直接取值。

三种方式对比

方式	语法	推荐度
<code>%</code> 格式化	<code>"%s %d" % (val1, val2)</code>	旧式，看得懂就行
<code>format()</code>	<code>"{} {}".format(val1, val2)</code>	中间代，了解即可
f-string	<code>f"{val1} {val2}"</code>	主力，推荐